

PilotMaster: A Modular Retrieval Intelligence Platform for Retrieval Engineering, Observability, and Benchmarking

Adhi Shakthi
shakadhi7@gmail.com

Abstract—Retrieval-Augmented Generation (RAG) systems have emerged as an effective approach for grounding large language models using external knowledge. However, existing RAG applications primarily emphasize document question answering while providing limited support for retrieval engineering, execution observability, and systematic evaluation of retrieval pipelines. This paper presents PilotMaster, a modular Retrieval Intelligence Platform that unifies document intelligence, retrieval observability, and experimental benchmarking through a shared retrieval framework. At the core of the platform is PilotCore, a reusable orchestration framework that centralizes document ingestion, retrieval execution, response generation, evaluation, tracing, and benchmarking within a unified pipeline. PilotMaster comprises three integrated applications: DocPilot for grounded document question answering, TracePilot for retrieval observability and execution diagnostics, and GaugePilot for controlled experimentation and comparative benchmarking of retrieval configurations. The platform supports heterogeneous document ingestion, configurable lexical, dense, and hybrid retrieval strategies, optional reranking and query enhancement techniques, integrated retrieval-centric evaluation, deterministic benchmark analysis, and optional AI-assisted engineering reports. By consolidating retrieval engineering workflows within a shared execution framework, PilotMaster enables reproducible experimentation while maintaining consistency between operational document intelligence and experimental evaluation. Experimental results demonstrate the applicability of the platform for systematic retrieval engineering across diverse retrieval configurations and heterogeneous document collections.

Keywords— Retrieval-Augmented Generation, Retrieval Intelligence, Retrieval Engineering, Document Intelligence, Information Retrieval, Retrieval Observability, Benchmarking, Large Language Models.

I. INTRODUCTION

Large Language Models (LLMs) have significantly advanced natural language understanding and generation across a wide range of applications. Despite their strong reasoning and language capabilities, these models remain constrained by static training data and are susceptible to hallucinations when responding to knowledge-intensive queries. Retrieval-Augmented Generation (RAG) addresses this limitation by grounding language model responses using externally retrieved information, enabling document-aware question answering, enterprise knowledge management, and domain-specific conversational systems. Consequently, RAG has become one of the most widely adopted paradigms for integrating large language models with private and continually evolving knowledge sources.

Although substantial research has focused on improving retrieval algorithms and generation quality, comparatively less attention has been devoted to the engineering workflow required to develop, analyze, and evaluate retrieval pipelines. In many existing RAG systems, retrieval remains a black-box process in which only the final generated response is exposed to the user while intermediate retrieval decisions, ranking behavior, contextual grounding, and execution metadata remain largely inaccessible. This lack of observability makes it difficult to diagnose retrieval failures, compare alternative retrieval strategies, understand the impact of reranking and query enhancement techniques, or systematically optimize retrieval performance. As retrieval pipelines continue to grow in complexity, effective retrieval engineering increasingly requires execution transparency, reproducible experimentation, and comprehensive evaluation beyond answer quality alone.

Existing document intelligence platforms primarily emphasize document ingestion and question answering, while retrieval observability and benchmarking are typically addressed using separate tools or ad hoc evaluation workflows. Similarly, many retrieval evaluation frameworks operate independently of production retrieval systems, resulting in duplicated implementations and experimental pipelines that do not accurately reflect operational behavior. This separation complicates comparative experimentation, reduces reproducibility, and limits the ability to translate benchmark findings into practical improvements for deployed retrieval systems.

To address these challenges, this paper presents PilotMaster, a modular Retrieval Intelligence Platform that unifies document intelligence, retrieval observability, and experimental benchmarking through a shared retrieval framework. The platform is built upon PilotCore, a reusable orchestration framework that centralizes document ingestion, retrieval execution, response generation, evaluation, tracing, and benchmarking within a unified execution pipeline. Three specialized applications are constructed upon this common infrastructure: DocPilot provides grounded document question answering, TracePilot enables retrieval observability through execution tracing and diagnostic analysis, and GaugePilot supports controlled experimentation and comparative benchmarking of retrieval configurations. By ensuring that all applications execute the same underlying retrieval pipeline, the platform maintains consistency between operational document intelligence and experimental evaluation while minimizing duplicated imple-

mentation.

The primary contributions of this work are threefold. First, we present a modular retrieval engineering platform that integrates document intelligence, observability, and benchmarking through a shared execution framework. Second, we introduce an integrated retrieval evaluation methodology that combines retrieval-centric metrics, execution tracing, deterministic benchmarking, and comparative visualization within a unified workflow. Third, we demonstrate the applicability of the proposed platform through configurable retrieval experimentation across heterogeneous document collections using lexical, dense, and hybrid retrieval strategies together with optional reranking and query enhancement techniques. The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system architecture. Section IV describes the implementation details. Section V presents the experimental evaluation. Section VI discusses architectural considerations, limitations, and future work. Section VII concludes the paper.

II. RELATED WORK

Retrieval-Augmented Generation (RAG) has emerged as a widely adopted paradigm for grounding large language models using external knowledge sources. Early RAG frameworks integrated dense retrieval with sequence generation to improve factual accuracy and reduce hallucinations by conditioning language models on retrieved documents. Subsequent research has expanded retrieval methodologies through hybrid retrieval, query rewriting, reranking, and advanced retrieval optimization techniques, improving retrieval effectiveness across a broad range of knowledge-intensive tasks. While these approaches primarily focus on retrieval algorithms and answer quality, comparatively less attention has been devoted to the engineering infrastructure required to develop, analyze, and evaluate retrieval pipelines.

Document intelligence platforms have similarly evolved to support semantic document search, document question answering, and enterprise knowledge management over heterogeneous document collections. These systems typically provide document ingestion, indexing, and retrieval capabilities while exposing natural language interfaces for interacting with uploaded documents. Although highly effective for end-user applications, they generally treat retrieval as an internal implementation detail, offering limited visibility into intermediate retrieval behavior, execution diagnostics, or retrieval quality analysis. Consequently, understanding why retrieval succeeds or fails often remains difficult from the perspective of retrieval engineers.

Recent work has also emphasized the importance of retrieval evaluation and observability. Evaluation frameworks have introduced metrics for assessing faithfulness, grounding, retrieval relevance, and answer quality, while observability platforms provide execution tracing, latency analysis, and pipeline diagnostics for language model applications. Likewise, benchmarking frameworks have enabled comparative evaluation of different retrieval configurations and language

models. However, these capabilities are frequently implemented as independent tools or offline evaluation pipelines that operate separately from production retrieval systems, resulting in duplicated implementations and experimental workflows that may not accurately represent operational behavior.

PilotMaster differs from existing approaches by integrating document intelligence, retrieval observability, and experimental benchmarking within a unified retrieval engineering platform. Rather than treating retrieval, evaluation, tracing, and benchmarking as independent subsystems, the proposed architecture centralizes these capabilities within the shared PilotCore framework, allowing DocPilot, TracePilot, and GaugePilot to execute the same retrieval pipeline. This unified design enables consistent retrieval behavior across operational document intelligence, execution observability, and controlled experimentation while supporting reproducible retrieval engineering through a common execution framework.

III. SYSTEM ARCHITECTURE

A. Platform Overview

PilotMaster is a modular Retrieval Intelligence Platform designed to support the complete engineering lifecycle of Retrieval-Augmented Generation (RAG) systems. The platform integrates document intelligence, retrieval observability, experimentation, and benchmarking through a shared framework, PilotCore, which provides common retrieval infrastructure and runtime services to multiple specialized applications.

Unlike conventional RAG applications that tightly couple retrieval and answer generation within a single workflow, PilotMaster separates retrieval engineering concerns into reusable components. This architectural separation enables independent development of document intelligence, observability, and benchmarking capabilities while avoiding duplication of retrieval logic across the platform.

PilotMaster consists of three primary applications: DocPilot, TracePilot, and GaugePilot. DocPilot provides grounded question answering over user-provided documents. TracePilot captures detailed execution traces and retrieval diagnostics to improve transparency and facilitate the debugging of retrieval pipelines. GaugePilot provides an isolated environment for experimentation, benchmarking, and comparative evaluation of retrieval configurations. All three applications are built upon the shared services provided by PilotCore.

To support both operational usage and retrieval experimentation, PilotMaster provides two operating modes. Production Mode offers document intelligence through DocPilot together with retrieval observability through TracePilot for day-to-day interaction with indexed document collections. The Experimental Mode extends this functionality by introducing GaugePilot, enabling controlled experimentation with retrieval strategies, reranking methods, query enhancement techniques, deterministic benchmarking, and optional AI-assisted engineering analysis.

A key architectural design decision is the separation of the operational state from the shared retrieval infrastructure. The Production and Experimental modes maintain independent

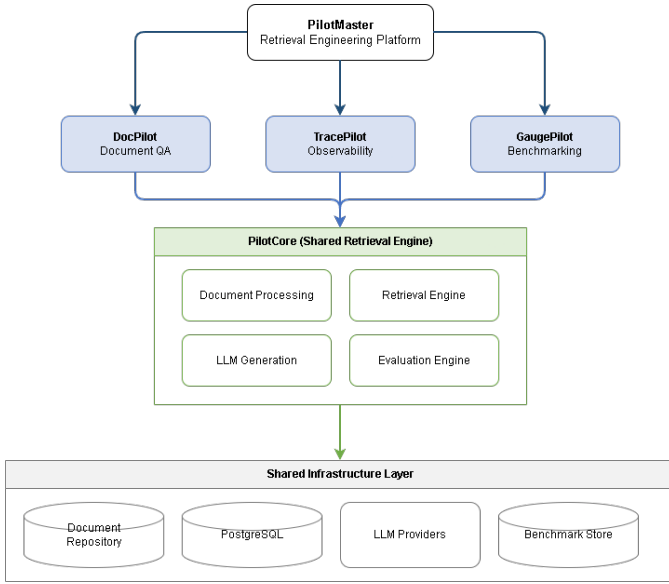


Fig. 1. Overall architecture of the proposed PilotMaster retrieval engineering platform

conversations and execution traces, ensuring that experimentation does not interfere with production workflows. At the same time, both modes reuse the same indexed document repository and vector storage, eliminating redundant document ingestion while guaranteeing that experiments operate over an identical knowledge base. This separation enables reproducible retrieval evaluation without duplicating storage or indexing resources.

B. PilotCore Framework

PilotCore is the shared retrieval orchestration framework that forms the architectural foundation of PilotMaster. Rather than implementing retrieval workflows independently within each application, PilotCore centralizes the execution of retrieval pipelines, evaluation, tracing, and benchmarking into a reusable framework that serves DocPilot, TracePilot, and GaugePilot.

The framework is responsible for coordinating the entire retrieval lifecycle. Beginning with a user query, PilotCore optionally performs query enhancement techniques such as query rewriting and multi-query generation before executing lexical, dense, or hybrid retrieval strategies. The retrieved candidates may then undergo Reciprocal Rank Fusion (RRF), deduplication, and optional cross-encoder reranking before being assembled into contextual prompts for grounded response generation.

Beyond retrieval execution, PilotCore evaluates every pipeline invocation using multiple retrieval-centric metrics, including retrieval quality, semantic grounding, faithfulness, semantic query coverage, answerability, context sufficiency, and abstention detection. These evaluation results become first-class artifacts within the execution pipeline rather than external post-processing outputs.

PilotCore also provides integrated observability through trace creation, telemetry collection, and execution lineage

recording. Retrieval metadata, ranking information, evaluation metrics, latency measurements, and execution events are captured throughout pipeline execution and may be forwarded to TracePilot for visualization and diagnostic analysis.

The same framework also serves as the execution engine for GaugePilot. Benchmark experiments invoke the identical retrieval pipeline used during document question answering, ensuring that comparative evaluations measure the behavior of the production retrieval workflow rather than a separate benchmarking implementation. This architectural reuse guarantees consistency between operational document intelligence and experimental evaluation while minimizing duplication across the platform.

C. Pipeline Execution Model

The execution model of PilotCore is centered around a unified pipeline responsible for orchestrating the complete retrieval lifecycle. Rather than maintaining separate execution paths for document question answering, observability, and benchmarking, all major applications within PilotMaster invoke the same underlying pipeline. This design ensures that production document interactions, retrieval diagnostics, and experimental evaluations are performed over an identical execution workflow.

Pipeline execution begins with the initialization of a trace object that uniquely identifies the request and records execution metadata throughout the retrieval lifecycle. Depending on the active experiment configuration, the incoming user query may optionally undergo one or more query enhancement stages, including query rewriting and multi-query generation, before retrieval is performed.

Retrieval is performed using one of three configurable strategies: BM25-based lexical retrieval, FAISS-based dense vector retrieval, or hybrid retrieval combining both approaches. During hybrid retrieval, candidate results from lexical and dense retrieval are merged using Reciprocal Rank Fusion (RRF) to produce a unified ranked candidate set. Retrieved chunks may subsequently undergo de-duplication and optional cross-encoder re-ranking before final context selection.

Following retrieval, the selected context is incorporated into a prompt construction stage and provided to the configured language model for grounded response generation. The resulting response, together with the retrieved evidence, forms the complete execution record for the pipeline.

Once response generation has completed, the PilotCore performs a comprehensive evaluation of the pipeline output. Retrieval relevance, semantic grounding, faithfulness, semantic query coverage, answerability, context sufficiency, and abstention behavior are computed directly from the executed pipeline, ensuring that evaluation remains tightly coupled with retrieval execution rather than operating as an external analysis step.

Throughout execution, telemetry events, retrieval lineage, latency measurements, ranking information, evaluation metrics, and execution spans are continuously recorded. When tracing is enabled, these execution artifacts are emitted to

TracePilot, enabling detailed visualization, replay, and diagnostic analysis without requiring modifications to the retrieval pipeline itself.

By exposing a single execution model across DocPilot, TracePilot, and GaugePilot, PilotCore guarantees that production document intelligence, retrieval observability, and benchmarking all operate over the same implementation. This architectural consistency minimizes duplicated logic while ensuring that experimental findings directly reflect the behavior of the operational retrieval pipeline.

D. DocPilot

DocPilot is the document intelligence application within the PilotMaster platform. It enables users to perform grounded question answering over an uploaded document while leveraging the shared execution capabilities provided by PilotCore. Rather than implementing its own retrieval pipeline, DocPilot delegates retrieval execution, response generation, evaluation, and tracing to the underlying PilotCore framework.

The primary responsibility of DocPilot is to provide a document-centric question answering workflow. A user uploads a document, which is processed through the shared ingestion pipeline before being embedded and indexed for retrieval. Once indexing is complete, user queries are executed through PilotCore’s unified pipeline using an experiment configuration determined by the selected operating mode and retrieval settings. The resulting grounded response is then returned together with the corresponding execution trace.

DocPilot is available in both Production and Experimental operating modes. Chat sessions are managed independently within DocPilot, while each query execution generates a unique PilotCore trace that records the complete retrieval and generation workflow. Although the two operating modes expose different user capabilities, both delegate execution to the same underlying PilotCore pipeline through mode-specific experiment configurations.

By delegating pipeline execution to PilotCore, DocPilot remains responsible for document interaction and grounded question answering while avoiding duplication of retrieval logic. Consequently, improvements to retrieval strategies, ranking methods, evaluation components, or tracing infrastructure within PilotCore become immediately available to DocPilot without requiring application-specific implementation changes.

E. TracePilot

TracePilot is the retrieval observability application within the PilotMaster platform. It provides visibility into the execution behavior of retrieval pipelines by collecting, storing, and presenting detailed execution traces generated by PilotCore. Rather than participating directly in retrieval execution, TracePilot serves as an observability layer that enables inspection, diagnosis, and analysis of pipeline behavior.

For every pipeline invocation, PilotCore creates a unique execution trace containing metadata collected throughout the

retrieval lifecycle. Trace information includes query metadata, retrieval lineage, retrieved chunks, ranking information, latency measurements, evaluation results, execution spans, and response metadata. These execution artifacts are emitted by PilotCore and ingested into TracePilot, where they are persisted for subsequent inspection and analysis.

TracePilot enables developers to examine individual pipeline executions in detail, allowing retrieval decisions to be understood beyond the final generated response. By exposing intermediate retrieval stages together with evaluation metrics, the platform facilitates identification of retrieval failures, ineffective ranking behavior, insufficient contextual grounding, and other pipeline-level issues that are difficult to diagnose through answer inspection alone.

Because TracePilot consumes execution artifacts produced by the same PilotCore pipeline used for document question answering and benchmarking, observability remains tightly coupled to operational behavior. This architectural reuse ensures that diagnostic insights accurately reflect the behavior of the production retrieval workflow without requiring a separate instrumentation pipeline or duplicate execution logic.

F. GaugePilot

GaugePilot is the experimentation and benchmarking application within the PilotMaster platform. Available exclusively in Experimental Mode, it provides a dedicated environment for systematically evaluating Retrieval-Augmented Generation (RAG) pipeline configurations without affecting production document intelligence workflows. Rather than implementing an independent execution pipeline, GaugePilot reuses the shared PilotCore framework, ensuring that benchmark results accurately reflect the behavior of the operational retrieval system.

GaugePilot supports controlled experimentation across configurable retrieval strategies, reranking methods, query enhancement techniques, and language models. Users define benchmark configurations, upload evaluation documents, specify evaluation questions, and execute benchmark runs through PilotCore’s unified pipeline. Because both document question answering and benchmarking invoke the same execution framework, experimental results remain directly representative of production retrieval behavior.

For every benchmark run, GaugePilot aggregates deterministic pipeline metrics, including average ranking performance, faithfulness estimation, semantic grounding, retrieval quality, semantic query coverage, and end-to-end latency. These metrics are consolidated into comparative leaderboards that rank competing retrieval configurations using identical evaluation criteria, enabling objective comparison across retrieval strategies, rerankers, and query enhancement techniques.

Beyond leaderboard generation, GaugePilot provides a comprehensive visualization suite for exploratory retrieval analysis. Comparative visualizations include scatter plots, radar charts, metric heatmaps, parallel coordinate plots, ranking distributions, metric distributions, correlation matrices, Pareto frontier

analysis, and performance profiles. Collectively, these visualizations expose trade-offs between retrieval effectiveness, answer quality, semantic grounding, coverage, and computational latency, enabling engineers to identify strengths and weaknesses across competing retrieval configurations.

A distinguishing architectural feature of GaugePilot is the separation of deterministic benchmarking from optional AI-assisted engineering analysis. Benchmark execution, metric computation, leaderboard generation, and visualization are completed entirely through deterministic evaluation, allowing benchmark results to become immediately available upon experiment completion. AI-generated engineering insight and recommendation reports are produced only upon explicit user request, preserving rapid benchmark execution while providing optional high-level interpretation of experimental outcomes.

By combining configurable experimentation, deterministic evaluation, interactive visualization, and optional AI-assisted engineering analysis over a shared execution pipeline, GaugePilot enables reproducible retrieval benchmarking and systematic retrieval engineering. The platform further supports comparative analysis across heterogeneous document domains, providing a foundation for studying how retrieval configurations behave on legal documents, medical records, research papers, source code, OCR-derived documents, and other document collections.

G. Architectural Design Rationale

The architecture of PilotMaster is guided by the principle of maximizing reuse while minimizing duplication across retrieval engineering workflows. Rather than implementing separate retrieval, evaluation, tracing, and benchmarking pipelines for individual applications, these responsibilities are centralized within PilotCore and exposed through a shared execution framework. This approach enables multiple applications to leverage identical retrieval behavior while remaining independently extensible.

A central design objective of PilotMaster is to maintain consistency between operational document intelligence and experimental evaluation. Both DocPilot and GaugePilot execute the same underlying PilotCore pipeline, ensuring that benchmark results directly reflect the behavior of the production retrieval workflow. Consequently, improvements introduced within PilotCore automatically benefit document intelligence, retrieval observability, and benchmarking without requiring application-specific modifications or duplicate implementations.

PilotMaster further separates operational workflows from experimental activities through distinct operating modes. Production and Experimental modes maintain independent conversational histories and execution traces while reusing the same indexed document repository and retrieval infrastructure. This separation enables controlled experimentation without disrupting production interactions and eliminates redundant document ingestion and indexing.

Another significant architectural decision is the decoupling of deterministic benchmarking from AI-assisted engineering analysis. Benchmark execution, metric computation,

and leaderboard generation are completed independently of language model inference, allowing experimental results to be produced immediately after benchmark completion. AI-generated engineering reports are executed only upon explicit user request, reducing latency and computational overhead while preserving access to detailed post-hoc analysis when required.

Collectively, these architectural decisions establish PilotMaster as a unified retrieval engineering platform rather than a collection of independent applications. By combining a shared orchestration framework, reusable execution pipeline, integrated observability, and systematic experimentation, the platform provides a consistent foundation for developing, analyzing, and evaluating Retrieval-Augmented Generation systems.

IV. IMPLEMENTATION

A. Document Ingestion

The document ingestion pipeline is responsible for transforming heterogeneous source documents into retrieval-ready knowledge representations that can be consumed by the shared PilotCore retrieval framework. Rather than implementing document processing independently within individual applications, PilotCore centralizes ingestion into a unified pipeline that prepares documents for document intelligence, retrieval observability, and experimental benchmarking. Consequently, every document ingested by the platform undergoes an identical pre-processing workflow irrespective of the consuming application or operating mode.

The ingestion pipeline is designed to accommodate a diverse range of document formats while producing a consistent internal representation suitable for downstream retrieval. Supported inputs include PDF documents, Microsoft Word documents, presentation slides, spreadsheets, images, plain-text and Markdown documents, structured data files, and multiple programming language source files. Despite differences in document structure and content, all inputs are normalized into a common sequence of retrieval units enriched with structural and contextual metadata before indexing.

To maximize retrieval quality while maintaining implementation consistency, PilotCore performs document ingestion through a sequence of well-defined stages comprising multi-stage document extraction, metadata enrichment, chunk generation, embedding construction, and dual-index creation. By performing these operations during ingestion rather than at query time, the platform shifts pre-processing overhead away from retrieval execution and ensures that subsequent retrieval workflows operate over precomputed dense and lexical retrieval structures.

B. Multi-Stage Document Extraction

PilotCore employs a multi-stage document extraction strategy to maximize text recovery across heterogeneous document types while minimizing unnecessary computational overhead. Rather than relying on a single extraction technique, the ingestion pipeline progressively applies increasingly robust

extraction methods only when sufficient textual content cannot be recovered during earlier stages. This design enables efficient processing of digitally generated documents while maintaining compatibility with scanned and image-based content.

For PDF documents, extraction begins with native digital text extraction using PyMuPDF. In addition to recovering textual content, the extractor analyzes page layout to identify document structure, including section headings, page numbers, and semantic section categories such as Introduction, Methods, Results, and Conclusion. If the extracted content falls below a predefined quality threshold, PilotCore automatically invokes Docling to perform structure-preserving document conversion and recover additional textual content. When both digital extraction and structured conversion prove insufficient, the pipeline performs page-level optical character recognition using PyTesseract after converting document pages into images. This progressive extraction cascade ensures that computationally expensive extraction techniques are employed only when required.

Document formats other than PDF files are processed using dedicated format-specific extractors. Microsoft Word documents, presentation slides, spreadsheets, plain-text documents, Markdown files, structured data files, source code, and images each follow specialized extraction routines that preserve document-specific structural information whenever possible. For example, spreadsheet rows retain worksheet information, presentation documents preserve slide numbers, source-code files record programming language metadata, and images undergo optical character recognition using PyTesseract to recover textual content. Despite differences in document structure, all extraction routines produce a common collection of normalized text sections suitable for downstream processing.

Throughout the extraction process, PilotCore enriches each extracted section with structural metadata describing its origin within the source document. Depending on the document type, metadata may include page numbers, slide numbers, worksheet identifiers, section titles, inferred document sections, programming language, document element type, OCR provenance, and source file information. This metadata accompanies every retrieval unit throughout indexing, retrieval, observability, and benchmarking, enabling downstream components to reason about document structure in addition to textual content.

C. Chunk Generation and Index Construction

Following document extraction, PilotCore transforms the normalized document sections into retrieval-ready units suitable for both dense and lexical retrieval. Rather than indexing complete documents, the platform partitions extracted content into smaller overlapping chunks, improving retrieval granularity while preserving sufficient contextual continuity for grounded response generation.

Chunk generation is implemented using a character-based sliding window strategy. The current implementation employs a default chunk size of 500 characters with an overlap of 80 characters between consecutive chunks. Instead of terminating

chunks at fixed character boundaries, the chunking algorithm attempts to preserve semantic coherence by adjusting chunk boundaries to the nearest sentence delimiter or new-line whenever possible. This boundary-aware strategy reduces fragmentation of natural language content while maintaining deterministic chunk sizes suitable for efficient indexing.

Each generated chunk is assigned an incremental identifier and inherits metadata from the corresponding extracted document section. In addition to the chunk text itself, every retrieval unit retains information describing its document source, document type, and logical position within the original document. Depending on the source format, metadata may include page numbers, slide numbers, worksheet identifiers, programming language, section titles, inferred document sections, OCR provenance, and document element type. This metadata accompanies each retrieval unit throughout indexing, retrieval, tracing, and benchmarking.

Following chunk generation, PilotCore converts every chunk into a dense vector representation using the `all-mpnet-base-v2` SentenceTransformer embedding model. Embeddings are generated as normalized vector representations and undergo additional L2 normalization prior to insertion into the vector index, ensuring consistent similarity computation during dense retrieval.

To support multiple retrieval strategies, PilotCore constructs complementary dense and lexical indexes during ingestion. Dense embeddings are stored within a per-user FAISS `IndexFlatIP` index configured with a 768-dimensional embedding space, while the corresponding chunk representations and metadata are persisted alongside the vector index. In parallel, a BM25 lexical index is constructed over the complete collection of indexed chunks using tokenized textual content. Both indexes are maintained independently for each user, enabling isolated document collections while supporting vector, lexical, and hybrid retrieval through a shared indexing infrastructure.

To maintain consistency between dense and lexical retrieval, PilotCore synchronizes both indexing structures whenever the underlying document collection is modified. Document insertion, deletion, and index reconstruction update the FAISS vector index and rebuild the corresponding BM25 index from the same collection of persisted document chunks. Consequently, dense and lexical retrieval operate over an identical knowledge base, providing a consistent foundation for hybrid retrieval strategies implemented within the retrieval pipeline.

D. Retrieval Pipeline

The retrieval pipeline constitutes the core execution workflow of PilotCore, orchestrating query processing, retrieval, ranking, response generation, evaluation, and observability through a unified runtime. Rather than maintaining separate execution paths for different applications, PilotCore exposes a single retrieval pipeline that is reused by document intelligence, retrieval observability, and benchmarking. This architectural design ensures that operational document interactions and experimental evaluations execute identical retrieval logic.

Pipeline execution begins with the creation of an execution trace that records the user query and serves as the central state object throughout the retrieval lifecycle. The active `ExperimentConfig` determines the retrieval strategy together with optional retrieval enhancements, allowing the same execution pipeline to operate under different experimental configurations without modifying the underlying implementation.

Prior to retrieval, PilotCore optionally performs query enhancement. Query rewriting reformulates the original user query into a retrieval-oriented representation, while multi-query retrieval generates multiple semantically related query variants that are executed independently. These capabilities are fully integrated into the retrieval pipeline and are activated through the experiment configuration. Additional enhancement options, including HyDE, query expansion, parent-child retrieval, contextual retrieval, GraphRAG, and context compression, are currently exposed through the experimental configuration interface but are reserved for future implementation.

PilotCore supports three retrieval strategies: lexical retrieval, dense vector retrieval, and hybrid retrieval. Lexical retrieval employs a BM25 index constructed during document ingestion and ranks candidate chunks according to lexical similarity. Dense retrieval embeds the incoming query using the same sentence embedding model employed during indexing before performing nearest-neighbor search over a FAISS `IndexFlatIP` vector index. Both retrieval strategies return retrieved chunks together with retrieval scores, ranking information, and metadata describing their provenance.

Hybrid retrieval combines dense and lexical retrieval within a unified execution workflow. Candidate documents are independently retrieved from both indexes before undergoing duplicate removal. The resulting ranked lists are subsequently merged using Reciprocal Rank Fusion (RRF), producing a single ranked candidate set that integrates evidence from both lexical and semantic retrieval without requiring direct comparison of heterogeneous scoring functions.

Following retrieval, PilotCore performs optional post-processing operations to improve retrieval quality. Duplicate retrieval units may be removed using a text-based deduplication strategy, while a configurable cross-encoder reranker reorders retrieved candidates according to query-document relevance. The reranker additionally computes confidence estimates and final ranking information that become part of the retrieval lineage recorded throughout pipeline execution.

The final ranked retrieval units are assembled into the retrieval context supplied to the response generation stage. Throughout execution, PilotCore continuously records retrieval metadata, ranking information, execution spans, latency measurements, and retrieval lineage within the active trace object. Upon completion of pipeline execution, these artifacts are reused by the response generation, evaluation, and observability components without requiring separate retrieval implementations, ensuring consistent behavior across all applications built upon the PilotCore framework.

E. Response Generation

Following context assembly, PilotCore generates the final grounded response using a centralized response generation module. Rather than coupling response generation with retrieval logic, the framework separates prompt construction from language model invocation, allowing retrieval, generation, and evaluation to remain modular components within the unified execution pipeline.

Prompt construction is performed using the retrieved context stored within the active execution trace. Retrieved chunks are concatenated into a single contextual representation and incorporated into one of two prompt templates depending on the nature of the user request. Broad information requests, such as summarization or explanation tasks, employ a document-oriented prompt template, whereas focused information requests use a question-answering template. Prompt selection is determined through a lightweight lexical heuristic based on the first token of the user query.

PilotCore currently integrates with the Groq inference platform for language model execution. The framework supports multiple configurable language models through a centralized model registry, while enforcing validation against a predefined set of supported models. During execution, the generated prompt is submitted as a chat completion request to the selected model, and the returned response is recorded as the final pipeline output.

Response generation is fully integrated with the tracing infrastructure. Upon completion, the generated response is stored within the active execution trace and accompanied by generation metadata, including the selected language model and response length. When trace emission is enabled, both the constructed prompt and generated response are forwarded together with retrieval, evaluation, and execution metadata to TracePilot, enabling complete end-to-end observability of the generation stage.

By isolating prompt construction, model selection, and language model invocation from the remainder of the retrieval pipeline, PilotCore maintains a modular generation subsystem that can accommodate multiple language models while preserving a consistent retrieval and evaluation workflow across all applications built upon the framework.

F. Evaluation Framework

PilotCore incorporates an integrated evaluation framework that assesses retrieval and response quality immediately following response generation. Rather than treating evaluation as an external post-processing step, the framework performs evaluation as part of the unified execution pipeline, ensuring that every completed retrieval task produces both a generated response and a corresponding set of retrieval-centric quality measurements.

The evaluation framework computes multiple complementary metrics that characterize different aspects of retrieval and generation performance. These include retrieval quality, semantic grounding, faithfulness estimation, semantic query coverage, answerability, context sufficiency, and abstention

behavior. Collectively, these metrics provide a comprehensive assessment of how effectively the retrieved evidence supports the generated response while measuring the overall effectiveness of the retrieval pipeline.

Evaluation results are attached directly to the active execution trace together with retrieval metadata, latency measurements, ranking information, and generation artifacts. Consequently, every pipeline invocation produces a complete execution record containing the original query, retrieved evidence, generated response, evaluation metrics, and execution telemetry. This unified representation enables downstream observability and benchmarking components to operate over standardized execution traces without requiring additional evaluation passes.

The evaluation framework also serves as the foundation for PilotCore’s benchmarking capabilities. During benchmark execution, identical evaluation metrics are computed for every experimental configuration, allowing retrieval strategies, reranking methods, and query enhancement techniques to be compared using a consistent set of quantitative measurements. By integrating evaluation directly into the execution workflow, PilotCore ensures that operational document interactions and experimental benchmarking rely on the same evaluation methodology, improving consistency and reproducibility across the platform.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

The experimental evaluation was conducted using the integrated benchmarking framework provided by GaugePilot. Rather than relying on external benchmarking scripts, benchmark experiments reuse the same PilotCore retrieval pipeline employed during operational document question answering. Consequently, benchmark results reflect the behavior of the production retrieval workflow under different retrieval configurations.

Each benchmark experiment is initiated through a single `ExperimentConfig`, which defines the retrieval configuration to be evaluated. The current implementation allows configuration of the language model, retrieval strategy, reranking model, and selected query enhancement techniques. Configurable retrieval strategies include lexical, dense vector, and hybrid retrieval, while reranking may be disabled or enabled using a selectable cross-encoder model. Query enhancement options exposed through the benchmark interface include query rewriting, HyDE, multi-query retrieval, and query expansion.

Evaluation documents are supplied through the shared document ingestion pipeline described in Section IV. Benchmark execution operates over the existing indexed document collection, with document preprocessing and indexing handled independently of the benchmarking workflow. Evaluation questions are provided as benchmark inputs and executed sequentially using the configured retrieval pipeline.

For each benchmark request, GaugePilot constructs a single experiment configuration and repeatedly invokes the shared

PilotCore pipeline once for every evaluation question. Each pipeline execution produces an execution trace containing retrieval results, generated responses, latency measurements, and evaluation metrics. Following completion of all evaluation questions, the collected metrics are aggregated into a configuration-level benchmark summary by computing average performance across the executed queries.

The benchmarking framework records multiple retrieval-oriented evaluation metrics, including faithfulness, semantic grounding, semantic query coverage, retrieval quality, execution latency, grounded response rate, and abstention rate. These aggregated metrics are subsequently used to generate deterministic leaderboards and comparative rankings across benchmarked retrieval configurations.

Upon completion of benchmark execution, GaugePilot persists the aggregated benchmark results together with the generated leaderboard and deterministic engineering analysis. In addition, the platform supports optional AI-generated engineering reports that may be produced independently after benchmark completion. By separating deterministic benchmark execution from optional AI-assisted interpretation, the framework enables immediate availability of quantitative benchmark results while allowing more detailed qualitative analysis to be generated on demand.

B. Evaluation Metrics

Benchmark evaluation is performed using the integrated evaluation framework implemented within PilotCore. Rather than relying on a single evaluation score, the framework computes multiple complementary metrics that collectively assess retrieval effectiveness, response grounding, semantic consistency, and execution efficiency. All evaluation metrics are computed immediately following response generation and attached to the execution trace before benchmark aggregation.

Faithfulness is estimated using a lexical overlap heuristic between the generated response and the retrieved document context. After removing common stop words, the framework computes the proportion of meaningful response terms that also appear within the retrieved document chunks. Responses matching predefined abstention phrases are treated as successful abstentions and assigned maximal faithfulness scores. In addition, a binary grounded indicator is derived from the extent of lexical overlap between the generated response and the retrieved context.

Semantic grounding is evaluated independently using dense semantic representations. The generated response and the concatenated retrieved context are embedded using the shared sentence embedding model, and semantic grounding is computed as the cosine similarity between their normalized embedding vectors. Similarly, semantic query coverage is computed by measuring the semantic similarity between the original user query and the retrieved document context, providing an estimate of how effectively retrieval captures the information required to answer the query.

Retrieval quality is computed using a composite heuristic that combines reranker confidence, retrieval source consensus,

reranker score margins, and lexical query-document relevance. The resulting composite score is normalized to produce a bounded retrieval quality metric that reflects both retrieval confidence and agreement between dense and lexical retrieval strategies.

The evaluation framework additionally estimates answerability and context sufficiency using lexical coverage between the user query and the retrieved document context. Based on predefined lexical coverage thresholds, each query is categorized according to its estimated answerability while the retrieved context is classified as sufficient, partially sufficient, or insufficient for supporting response generation. Abstention behavior is identified through heuristic phrase matching, allowing the framework to distinguish intentionally abstained responses from unsupported answer generation.

During benchmark execution, evaluation metrics are aggregated across all benchmark questions for each retrieval configuration. Configuration-level averages are computed for faithfulness, semantic grounding, semantic query coverage, retrieval quality, and execution latency, while grounded response rate and abstention rate are computed as proportions across all evaluated queries. These aggregated metrics subsequently form the basis for leaderboard generation, comparative visualization, and engineering analysis.

C. Benchmark Results

Benchmark experiments were conducted using nine retrieval pipeline configurations spanning lexical, dense vector, and hybrid retrieval strategies together with optional reranking and query enhancement techniques. Each configuration was evaluated using the same benchmark document and evaluation question set, ensuring that performance differences resulted solely from variations in retrieval pipeline configuration rather than differences in the underlying knowledge base or evaluation methodology.

For every benchmark run, GaugePilot aggregated faithfulness, semantic grounding, retrieval quality, semantic query coverage, and end-to-end execution latency across all evaluation questions. Configuration rankings were subsequently determined using an aggregate ranking methodology that combines relative performance across all evaluation metrics rather than relying on a single optimization objective. This approach enables balanced comparison between retrieval effectiveness and computational efficiency.

Fig. 2 presents the overall benchmark rankings obtained during experimentation. Among the evaluated configurations, dense vector retrieval using the GPT-OSS-120B language model achieved the highest overall performance, demonstrating the strongest balance between semantic grounding, faithfulness, retrieval effectiveness, and execution latency. Lexical retrieval produced the lowest execution latency, whereas several hybrid configurations achieved competitive retrieval performance at the cost of increased computational overhead. Configurations employing multiple query enhancement techniques and reranking generally exhibited higher latency due

RANK	CONFIGURATION	AVG RANK	FAITHFULNESS	GROUNDING	RETRIEVAL	COVERAGE	LATENCY
1	gpt-oss-120b vector none Default	3.2	0.827	0.705	0.129	0.705	561 ms
2	lama-3.3-70b vector none Default	4.8	0.780	0.721	0.129	0.705	583 ms
3	gpt-oss-120b lexical tinybert Default	5	0.783	0.630	0.455	0.647	440 ms
4	gpt-oss-120b vector minilm Default	5	0.816	0.695	0.293	0.690	731 ms
5	gpt-oss-120b hybrid none Query Rewrite	5	0.806	0.700	0.200	0.689	694 ms
6	gpt-oss-120b hybrid minilm Default	5.2	0.756	0.699	0.343	0.693	979 ms
7	gpt-oss-120b hybrid none Default	5.2	0.747	0.701	0.179	0.703	632 ms
8	gpt-oss-120b hybrid tinybert Multi Query Query Rewrite	5.4	0.827	0.692	0.303	0.673	14256 ms
9	gpt-oss-120b lexical none Default	6.2	0.793	0.613	0.136	0.619	334 ms

Fig. 2. Overall benchmark leaderboard comparing the nine evaluated retrieval configurations.

to the additional retrieval and ranking operations performed during pipeline execution.

Beyond numerical rankings, the benchmark framework generated a comprehensive suite of comparative visualizations, including scatter plots, radar profiles, metric heatmaps, correlation matrices, Pareto frontier analysis, ranking distributions, and parallel coordinate plots. These visualizations provide complementary perspectives on retrieval behavior and expose trade-offs between retrieval quality, semantic grounding, faithfulness, coverage, and latency across competing retrieval configurations.

In addition to deterministic benchmarking, GaugePilot supports optional AI-assisted engineering analysis that operates independently of benchmark execution. Rather than influencing benchmark results, AI-generated insight and recommendation reports analyze the completed benchmark outputs to summarize observed strengths, weaknesses, engineering trade-offs, and potential optimization opportunities. Because report generation is explicitly decoupled from deterministic benchmarking, quantitative benchmark results remain immediately available after experiment completion while qualitative engineering analysis can be generated or regenerated on demand without requiring benchmark re-execution.

D. Comparative Retrieval Analysis

The benchmark results demonstrate that retrieval performance is governed by trade-offs between semantic accuracy, retrieval effectiveness, and computational efficiency rather than by a single dominant configuration. Figure 3 presents the normalized performance profiles of the highest-ranked retrieval pipelines across the five evaluation metrics, while Figure 4 illustrates the corresponding metric trade-offs using parallel coordinates.

Dense vector retrieval using the GPT-OSS-120B model achieved the highest overall benchmark ranking, consistently demonstrating strong faithfulness and semantic grounding while maintaining relatively low execution latency. In contrast, lexical retrieval exhibited the fastest response times but produced lower grounding and retrieval quality scores,

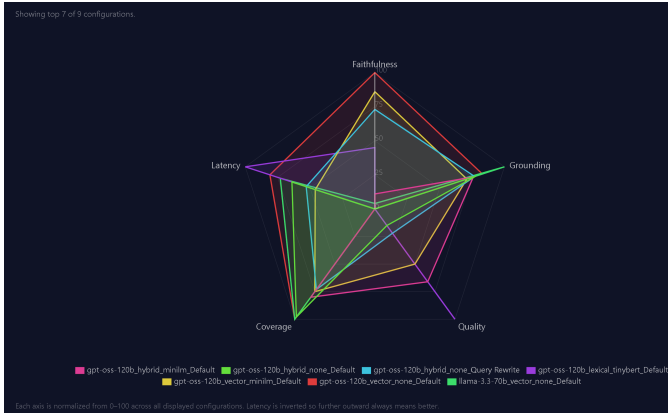


Fig. 3. Normalized performance profiles of the top-ranked retrieval configurations across the five evaluation metrics.

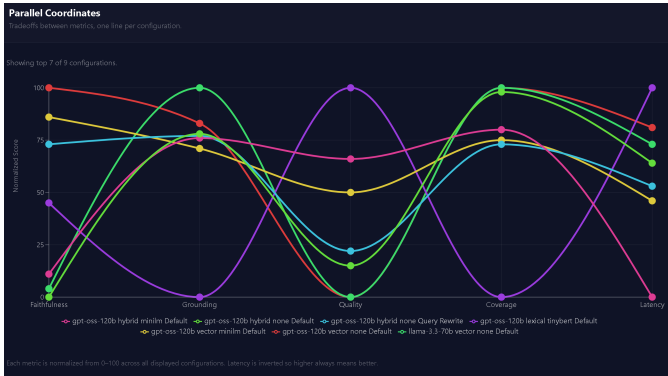


Fig. 4. Parallel coordinates illustrating metric trade-offs across the evaluated retrieval configurations.

highlighting the limitations of purely keyword-based retrieval for semantically complex queries.

Hybrid retrieval configurations incorporating reranking and query enhancement generally improved retrieval quality and semantic grounding; however, these improvements were accompanied by increased computational overhead. In particular, the configuration combining hybrid retrieval, TinyBERT reranking, Multi-Query Retrieval, and Query Rewrite achieved high faithfulness but incurred the highest execution latency due to the cumulative cost of multiple retrieval and reranking stages.

The visualizations further demonstrate that no individual retrieval configuration simultaneously optimizes all evaluation metrics. Instead, each pipeline represents a different balance between retrieval accuracy, semantic coverage, execution latency, and computational cost. These observations reinforce the need for configurable retrieval pipelines capable of adapting to application-specific requirements rather than relying on a single static retrieval strategy.

E. AI-Assisted Benchmark Analysis

Beyond quantitative benchmarking, GaugePilot incorporates an AI-assisted analysis module that automatically interprets

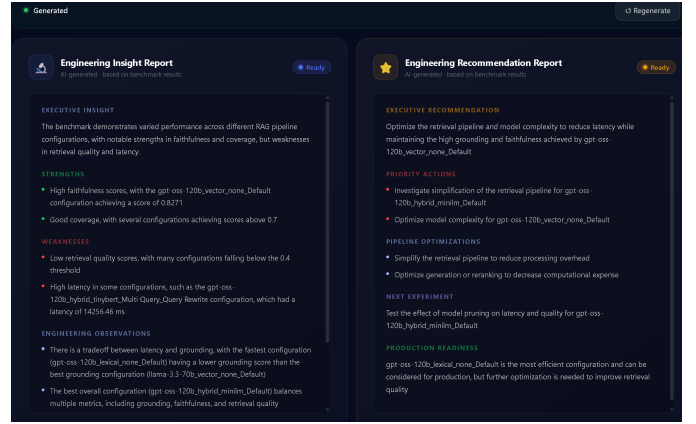


Fig. 5. Representative AI-generated engineering report produced by GaugePilot

benchmark outcomes and generates engineering-oriented reports. Following completion of a benchmark run, users may optionally generate natural-language insight and recommendation reports summarizing the strengths, weaknesses, trade-offs, and optimization opportunities observed across all evaluated retrieval configurations.

Unlike the deterministic benchmarking pipeline, AI-assisted report generation is intentionally decoupled from benchmark execution. This architectural decision ensures that quantitative benchmark results remain immediately available after experiment completion while allowing computationally intensive language model analysis to be performed independently and regenerated on demand without requiring benchmark re-execution.

Figure 5 illustrates a representative AI-generated engineering analysis produced by GaugePilot. The generated reports identify high-performing retrieval configurations, highlight trade-offs between faithfulness, semantic grounding, retrieval quality, and execution latency, and recommend candidate configurations for production deployment or future experimentation. By transforming numerical evaluation metrics into actionable engineering guidance, the analysis module reduces manual interpretation effort and supports more informed retrieval engineering decisions.

VI. DISCUSSION

A. Architectural Design Decisions

PilotMaster was designed around the principle of separating retrieval engineering concerns into modular, reusable components rather than tightly coupling document ingestion, retrieval, evaluation, observability, and benchmarking within a single application. This architectural separation enables individual capabilities to evolve independently while sharing a common retrieval infrastructure through the PilotCore framework.

A central design decision is the reuse of a unified execution pipeline across all platform applications. Rather than maintaining independent implementations for document question answering, retrieval observability, and benchmarking, DocPilot,

TracePilot, and GaugePilot all invoke the same underlying retrieval pipeline provided by PilotCore. Consequently, improvements to retrieval algorithms, ranking strategies, evaluation methods, or tracing mechanisms automatically become available throughout the platform without requiring application-specific modifications.

Another important architectural decision is the separation of operational state from shared retrieval infrastructure. Production and Experimental modes maintain independent conversations, execution traces, and application state while operating over the same indexed document collection. This separation allows retrieval experiments to be conducted without affecting production workflows while avoiding unnecessary duplication of document ingestion and indexing resources.

PilotMaster also separates deterministic retrieval evaluation from optional AI-assisted engineering analysis. Benchmark execution, metric computation, leaderboard generation, and comparative analysis are performed deterministically using the shared evaluation framework. AI-generated engineering reports are invoked only upon explicit request, reducing benchmark execution latency while preserving the ability to generate detailed qualitative interpretations of experimental results.

Collectively, these architectural decisions emphasize modularity, implementation reuse, reproducibility, and consistency across retrieval engineering workflows. By consolidating document intelligence, observability, and benchmarking around a shared execution framework, PilotMaster minimizes duplicated logic while ensuring that experimental evaluation remains representative of operational retrieval behavior.

B. Engineering Trade-offs

The architectural design of PilotMaster prioritizes modularity and implementation reuse over application-specific optimization. By centralizing document ingestion, retrieval, evaluation, tracing, and benchmarking within the shared PilotCore framework, the platform minimizes duplicated logic and ensures consistent behavior across DocPilot, TracePilot, and GaugePilot. However, this architectural consistency also implies that modifications to the shared execution pipeline affect all dependent applications, requiring careful consideration when introducing new retrieval capabilities or evaluation methods.

The platform also balances retrieval effectiveness against computational efficiency. Advanced retrieval techniques such as query rewriting, multi-query retrieval, and cross-encoder reranking can improve retrieval performance for certain workloads but introduce additional computational overhead and increased response latency. Rather than prescribing a single retrieval configuration, PilotMaster exposes these techniques as configurable pipeline components, enabling retrieval pipelines to be tailored according to application-specific performance requirements.

Another important trade-off concerns evaluation fidelity and computational cost. PilotCore computes multiple retrieval-oriented evaluation metrics during pipeline execution, allowing benchmark experiments and observability analyses to reuse the

same evaluation framework. While integrated evaluation provides comprehensive diagnostic information for every pipeline execution, it also introduces additional processing beyond response generation alone. Separating deterministic evaluation from optional AI-assisted engineering analysis mitigates this overhead by ensuring that computationally expensive report generation is performed only when explicitly requested.

Finally, the platform balances resource efficiency with experimental reproducibility. Production and Experimental modes maintain independent conversations and execution traces while sharing the same indexed document repository and retrieval infrastructure. This design avoids redundant document ingestion and indexing while ensuring that experimental evaluations are performed over the identical knowledge base used during operational document question answering.

C. Current Limitations

Although PilotMaster provides a unified platform for document intelligence, retrieval observability, and benchmarking, several limitations remain.

The current document ingestion pipeline employs fixed-size character-based chunking with predefined overlap parameters. While this approach produces consistent retrieval units and simplifies indexing, it does not explicitly preserve semantic boundaries. Future work may incorporate semantic or token-aware chunking techniques to improve context preservation and retrieval accuracy.

The benchmarking framework relies primarily on lightweight embedding-based similarity measures and heuristic evaluation metrics to assess faithfulness, retrieval quality, semantic grounding, coverage, and answerability. Although this design enables efficient large-scale benchmarking without requiring additional language model evaluation for every response, it cannot fully capture the nuanced semantic judgments obtainable through human evaluation or advanced LLM-as-a-Judge methodologies.

Benchmark scalability is also constrained by the availability and usage limits of hosted language model inference services. Preliminary experiments involving 42 evaluation questions across every retrieval configuration generated several hundred inference requests, exceeding practical service quotas during a single benchmark session. Consequently, the experimental evaluation presented in this work employs a reduced benchmark set while preserving representative comparative analysis. Future implementations may address this limitation through distributed benchmark execution, batching strategies, or self-hosted language models.

Response generation currently utilizes two predefined prompt templates selected through a lightweight lexical heuristic. While effective for distinguishing document summarization from focused question answering, more adaptive prompt selection and dynamic prompt optimization strategies could further improve response quality across diverse information needs.

Finally, the current implementation executes retrieval pipelines synchronously and maintains retrieval indexes on

a single-node architecture. Although adequate for the scale of the presented experiments, larger document collections and concurrent multi-user deployments would benefit from asynchronous pipeline execution, distributed indexing, and horizontally scalable vector storage.

D. Future Work

Several directions remain for extending the capabilities of PilotMaster. Future work includes investigating semantic and token-aware chunking strategies that preserve document structure more effectively than fixed character-based segmentation. Additional retrieval enhancements, including contextual retrieval, graph-based retrieval, and context compression techniques, may further improve retrieval effectiveness across complex document collections.

The evaluation framework may also be extended with more advanced retrieval assessment methodologies and additional benchmark datasets spanning multiple document domains. Such extensions would enable broader evaluation of retrieval behavior while preserving the deterministic benchmarking workflow provided by GaugePilot.

From a systems perspective, future development may focus on improving scalability through batch embedding generation, distributed indexing, and support for larger document repositories. Expanding support for additional language model providers and retrieval backends would further increase the flexibility of the shared PilotCore framework while maintaining architectural consistency across document intelligence, observability, and benchmarking applications.

More broadly, PilotMaster establishes a foundation for unified retrieval engineering in which document intelligence, retrieval observability, and experimental benchmarking operate over a shared execution framework. Future enhancements will continue to build upon this modular architecture while preserving the reproducibility and implementation consistency that motivate the platform.

VII. CONCLUSION

This paper presented PilotMaster, a modular Retrieval Intelligence Platform that unifies document intelligence, retrieval observability, and experimental benchmarking within a shared retrieval engineering framework. By centralizing document ingestion, retrieval execution, evaluation, tracing, and benchmarking inside the reusable PilotCore framework, the platform enables multiple applications to operate over an identical retrieval pipeline while avoiding duplicated implementation. The resulting architecture provides consistent behavior across operational document question answering, execution observability, and controlled retrieval experimentation.

The proposed platform demonstrates that retrieval engineering extends beyond response generation alone and requires comprehensive support for experimentation, evaluation, and diagnostic analysis. Through the integration of DocPilot, TracePilot, and GaugePilot, PilotMaster provides an end-to-end environment for developing, analyzing, benchmarking, and optimizing Retrieval-Augmented Generation (RAG) systems. Experimental evaluation across multiple retrieval configurations

illustrates the usefulness of the platform for studying retrieval trade-offs while maintaining reproducible benchmarking and consistent evaluation methodologies.

By combining configurable retrieval pipelines, integrated observability, deterministic benchmarking, comparative visualization, and optional AI-assisted engineering analysis within a unified execution framework, PilotMaster establishes a practical foundation for systematic retrieval engineering. Future extensions involving advanced retrieval strategies, scalable indexing architectures, and richer evaluation methodologies can be incorporated into the shared PilotCore framework without altering the overall platform architecture, enabling the system to evolve while preserving consistency, reproducibility, and modularity across retrieval workflows.

VIII. PROJECT AVAILABILITY

The complete implementation of PilotMaster, including DocPilot, TracePilot, GaugePilot, and the shared PilotCore framework, is publicly available.

Live Application: <https://pilot-master.vercel.app>

GitHub Repository: PilotMaster GitHub Repository

REFERENCES

- [1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, 2020.
- [2] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of EMNLP*, 2020.
- [3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of EMNLP-IJCNLP*, 2019.
- [4] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [5] J. Johnson, M. Douze and H. Jégou, "Billion-scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [6] G. V. Cormack, C. L. A. Clarke and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," in *Proceedings of SIGIR*, 2009.
- [7] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv:1901.04085, 2019.
- [8] N. Thakur, N. Reimers, J. Daxenberger and I. Gurevych, "Augmented SBERT: Data Augmentation Method for Improving Bi-Encoders for Pairwise Sentence Scoring Tasks," arXiv:2010.08240, 2020.
- [9] E. S. Es et al., "RAGAs: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of EACL*, 2024.
- [10] R. Smith, "An Overview of the Tesseract OCR Engine," in *Proceedings of ICDAR*, 2007.
- [11] Artifex Software, "PyMuPDF Documentation," Available: <https://pymupdf.readthedocs.io/>.
- [12] IBM Research, "Docling: Document Processing for Generative AI," Available: <https://github.com/docling-project/docling>.